



Module 3: Generic data types

Overview

- Non-Generic vs Generic Solution
- Generic Collections
- Advanced topics:
 - Multiple Generic Parameters
 - Overloading Generics
 - Constraints
 - Other Generic Types, Casting and Inheritance

Generics Problem Statement

- General-purpose stack
- To store instances of various types

```
public class Stack
{
    object[] _items;
    public void Push(object item) {...}
    public object Pop() {...}
}
```

```
Stack stack = new Stack();
stack.Push("2");
stack.Push(1);
string text = (string)stack.Pop();
int number = (int)stack.Pop();
```

Specialized tools

- Each tool is suitable only for very specific use

```
public class Stack
{
    int[] _items;
    public void Push(int item) {...}
    public int Pop() {...}
}
```

```
public class Stack
{
    string[] _items;
    public void Push(string item) {...}
    public string Pop() {...}
}
```



Example of generic class

- Use the < and > brackets, enclosing a generic type parameter

```
public class Stack<T>
{
    T[] _items;
    public void Push(T item) {...}
    public T Pop() {...}
}
```



```
Stack<int> stack = new Stack<int>();
stack.Push(1);
stack.Push("2"); // syntax error
int number = stack.Pop();
```



default keyword

```
class MyClass
{
    int value1;           // constructor set default value 0
    void doIt()
    {
        int value2;      // unsigned variable
    }
}
```

- Initializing to default values

```
default(int)           // 0
default(bool)          // false
default(StringBuilder) // null
default(char)          // '\0'
```

- In generic types for initializing local variables

```
T item = default(T);
```

Generics

- Generics allow you to define type-safe classes without compromising type
 - Type safety
 - Performance
 - Productivity
 - Reusing code

Note:

- *from .Net 2.0*

Lab 03.1 - Generics - introduction to the topic



Example:

1. Creating a generic class Stack
2. problem No.1
3. problem No.2
4. Typed solution
5. Generic solution
6. Using generic class
7. Using default keyword

Generic Collections

System.Collections	System.Collections.Generic
HashTable	Dictionary<K,T>
-	LinkedList<T>
ArrayList	List<T>
Queue	Queue<T>
SortedList	SortedDictionary<K,T>
Stack	Stack<T>

Lab 03.2 - Array and Collections in .Net



Example:

1. Using Array
2. Using ArrayList
3. Using List<>
4. Using Queue<>
5. Using Stack<>
6. Using Dictionary<>

Overview

- Non-Generic vs Generic Solution
- Generic Collections
- **Advanced topics:**
 - **Multiple Generic Parameters**
 - **Overloading Generics**
 - **Constraints**
 - **Other Generic Types, Casting and Inheritance**

Multiple Generic Parameters

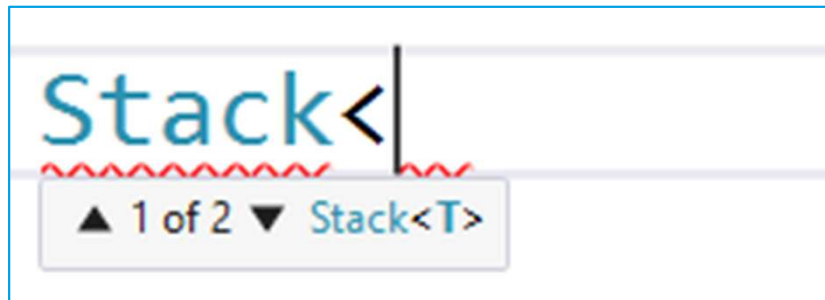
- A single type can define multiple generic-type parameters.

```
class Node<K, T>
{
    public K Key;
    public T Item;
}
```

Overloading generics

- Generic classes can be Overloaded

```
class Stack<T> { }  
class Stack<K,T> { }
```



Constraints for generic types

- Any type for generic arguments can be used by default
- Three types of constrains
 1. Derivation constraints from class or interface
 2. Default constructor constraint
 3. Reference/value type constraint constrains
 4. Notnull and unmanaged constraint $\geq$ C# 8.0

Derivation Constraints

- Generic type parameter must be derived from a particular base class or must be base class itself

```
public class LinkedList<K,T> where K : MyBaseClass  
{...}
```

- Generic type parameter must implement interface

```
public class LinkedList<K,T> where K : IComparable  
{...}
```

Note:

- *Some types are disallowed as a base class constraint: **Object**, **Array**, and **ValueType***

Constructor Constraint

- Parameter such that must support a public default constructor

```
class Node<K,T> where T : new()  
{...}
```


Reference/value type constraint constrains

- Value type constraint for int, a bool, and enum or any custom structure

```
public class MyClass<T> where T : struct {...}
```

- Reference type constraint

```
public class MyClass<T> where T : class {...}
```

>=C# 7.3

- Enum type constraint

```
public class MyClass<T> where T : Enum {...}
```

- Delegate type constraint

```
public class MyClass<T> where T : Delegate {...}
```

NotNull and unmanaged constraint >= C# 8.0

- Notnull constraint

```
public class MyClass<T> where T : notnull  
{...}
```

```
#nullable enable
```

```
MyClass<string> mc1 = new MyClass<string>(); // OK
```

```
MyClass<string?> mc2 = new MyClass<string?>(); // Compile error  
// 'string?' doesn't match 'nonnull' constraint.
```

Combining constraints

- You can combine the constructor constraint with derivation constraint

```
public class LinkedList<K,T>  
    where K : IComparable, new()  
{...}
```

Note:

- *Constructor constraint appears last in the constraint list*

Combining constraints

- Parameter must implement two interfaces

```
public class LinkedList<K,T>  
    where K : IComparable, IConvertible  
{...}
```

- First parameter must implement other interface than the second one

```
public class LinkedList<K,T>  
    where K : IComparable  
    where T : IConvertible  
{...}
```

Inheritance and Generics

- When deriving from a generic base class, you must provide a type argument

```
public class BaseClass<T>
{...}

public class SubClass : BaseClass<int>
{...}
```

- If the subclass is generic, instead of a concrete type argument, you can use the subclass generic type parameter

```
public class SubClass<T> : BaseClass<T>
{...}
```

Inheritance and Generics

- You must repeat any constraints defined at the base class level at the subclass level

```
public class BaseClass<T> where T : ISomeInterface  
{...}
```

```
public class SubClass<T> : BaseClass<T> where T : ISomeInterface  
{...}
```

Generic types

- You can define:
 - generic classes
 - generic abstract classes
 - generic interfaces
 - generic methods
 - generic abstract methods
 - generic delegates
 - generic collections
 - generic operator

Note:

- *These types behave like any other generic base type*

Generic Interface, abstract class and abstract method

- Generic Interface

```
public interface ISomeInterface<T>
{
    T SomeMethod(T t);
}
```

- Abstract class and abstract method

```
public abstract class BaseClass<T>
{
    public abstract T SomeMethod(T t);
}
```


Generic methods

- A method can define generic type parameters, specific to its execution scope

```
public class MyClass
{
    public void MyMethod<T>(T t){...}
}
```

- you can provide the type to use when calling method

```
MyClass obj = new MyClass();
obj.MyMethod<int>(3);
```

- or omitting the type specification

```
MyClass obj = new MyClass();
obj.MyMethod(3);
```

Generics and the .NET

- **Generic methods of System.Array**

```
public static void Sort<T>  
public static int  IndexOf<T>  
public static T[] FindAll<T>  
public static void ForEach<T>  
public static int  BinarySearch<T>  
public static T Find<T>  
...
```

Lab 03.3 - Parameters and Constraints



Example:

- Overloading generics
- Type aliasing
- Generic methods

Exercise - Generics



- Create a generic interface **IRepository<T>** that defines basic CRUD operations:
 - void **Add**(T item)
 - void **Remove**(T item)
 - T **GetById**(int id)
 - List<T> **GetAll**()
- Implement the **Repository<T>** class, which stores items in an internal list
- Create an **Employee** class with properties **Id**, **FirstName**, and **LastName**
- Demonstrate usage in the **Main()** method

Expected output:

```
Employees:  
1: John Doe  
2: Jane Smith
```

Exercise - Helper.Switch() generic method



Create class Helper and generic method Switch() that switch values of its two parameters

- a) Parameters can be of any type
- b) Parameters can be only of value type

Review

- Non-Generic vs Generic Solution
- Generic Collections
- Advanced topics:
 - Multiple Generic Parameters
 - Overloading Generics
 - Constraints
 - Other Generic Types, Casting and Inheritance